
Midterm Report

SERGE: AI Agent for Supply Chain Operations

A Ready-to-Use System Connecting Supply Chain
Infrastructure to Industry-Standard AI Agents

Louis Grignola Gauthier Schmitt
Maël Tellus Matteo Schwarz

Course: SE618-1 Logistics

Institution: Sirindhorn International Institute of Technology (SIIT)

Date: March 2026

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Objectives	3
1.3	Scope	4
2	Literature Review	5
2.1	Large Language Models	5
2.1.1	Scaling Laws and Emergence	5
2.1.2	Current Landscape	5
2.2	AI Agents: From ReAct to Modern Architectures	5
2.2.1	Foundational Work: ReAct	5
2.2.2	Beyond ReAct: Modern Agent Paradigms	6
2.2.3	Claude Code as a State-of-the-Art Agent	6
2.2.4	Tool Use in LLMs	6
2.3	Model Context Protocol	7
2.4	Supply Chain Management: Fragmentation and Visibility	7
2.5	AI and LLM Applications in Supply Chain (2023–2026)	8
2.5.1	Multi-Agent Supply Chain Systems	8
2.5.2	Diagnostic and Knowledge-Based Approaches	8
2.5.3	Emerging Paradigms	8
2.6	LLM-as-Judge Evaluation	8
2.7	Research Gaps	9
3	Methodology	11
3.1	Overall Approach	11
3.2	Software Architecture: Four Go Services	11
3.2.1	SRM: Supplier Relationship Management	11
3.2.2	WMS: Warehouse Management System	12
3.2.3	OMS: Order Management System	12
3.2.4	TMS: Transport Management System	12
3.3	MCP Integration Layer	13
3.4	AI Agent: Claude Code in a Dedicated Project	14
3.5	Use Cases	14
3.6	Evaluation: LLM-as-Judge Pipeline	15
3.6.1	Judge Evaluation Criteria	15
3.6.2	Evaluation Protocol	16
3.7	Development Timeline	16

4	Expected Outputs	17
4.1	What We Expect from the Agent	17
4.2	Project Deliverables	17
4.3	Research Contributions	17
4.4	Knowledge Gained	18
4.4.1	Supply Chain Domain	18
4.4.2	Software Engineering	18
4.4.3	AI and Agent Engineering	18
5	Progress Plan	19
5.1	Completed Work (Weeks 1–5)	19
5.2	Remaining Work (March 20 to April 15)	19
5.3	Risk Assessment	20

Introduction

Problem Statement

Modern supply chains rely on multiple enterprise systems operating in silos: Supplier Relationship Management (SRM), Warehouse Management (WMS), Order Management (OMS), and Transport Management (TMS). Each system is typically provided by a different vendor, accessed through a different interface, and stores its data independently. According to a GEODIS survey, **only 6% of companies report full supply chain visibility**, while nearly 80% have either no visibility or a restricted view [25].

In a B2B distribution context, this fragmentation creates a concrete operational problem:

Customer Pain Point

Problem: Operations managers spend **15–30 minutes per investigation** when diagnosing supply chain issues (delayed orders, stockouts, supplier problems). They must manually log into 3–4 separate systems, extract data from each, and mentally piece together the causal chain.

Consequences:

- Slow response to customer inquiries about order status
- Incomplete investigations: humans miss data points across systems
- Knowledge concentrated in senior staff who know “where to look”
- No structured audit trail of investigations

Each system “speaks a different language”: finance in ERP, orders in OMS, inventory in WMS, transport in TMS [26]. Without integration, cross-system queries require manual reconciliation, creating communication delays, errors, and inefficiencies that worsen at scale.

Objectives

The objective of this project is to design, implement, and evaluate **SERGE**, an AI agent that provides a **ready-to-use system** for connecting supply chain infrastructure to any industry-standard AI agent. Concretely, SERGE:

1. **Connects** the four supply chain systems (SRM, WMS, OMS, TMS) to AI agents through the Model Context Protocol (MCP), an open industry standard.
2. **Retrieves** data from any combination of systems in response to natural language queries.
3. **Reasons** across systems to identify root causes, correlations, and patterns (e.g., “this order is delayed because the supplier shipment is late, causing an inventory shortage”).
4. **Generates** structured diagnostic reports with source attribution.
5. **Remains strictly read-only:** SERGE never writes to the supply chain databases.

Because SERGE exposes supply chain data through the MCP standard, **any MCP-compatible agent** (Claude, GPT, Llama, Mistral, or custom agents) can plug into the supply chain infras-

tructure without any modification to the data layer. The system is designed as a bridge between existing supply chain software and the rapidly evolving ecosystem of AI agents.

Scope

We explicitly scope SERGE as a **retrieval and diagnostic** agent, not a predictive or decision-making one:

Capability	In Scope?	Description
Data retrieval	Yes	Query any of the 4 systems via natural language
Cross-system diagnostic	Yes	Connect data across SRM, WMS, OMS, TMS to find root causes
Report generation	Yes	Produce structured summaries and comparisons
Predictive analytics	No	Forecasting demand, predicting delays (future work)
Autonomous decisions	No	Reordering stock, rerouting shipments (out of scope)

This scoping is deliberate: in practice, companies deploy read-only AI assistants *before* trusting AI with write access. Our agent reflects this real-world adoption pattern.

Key Insight

B2B distribution is the ideal setting because it involves all four supply chain systems with high interdependency. A single delayed supplier shipment can cascade into stockouts, delayed customer orders, and carrier rebooking, which is exactly the kind of cross-system reasoning our agent targets.

Literature Review

Large Language Models

Large Language Models (LLMs) are the foundational technology powering modern AI agents. Built on the Transformer architecture introduced by Vaswani et al. [1], LLMs rely entirely on self-attention mechanisms to process input sequences in parallel, capturing long-range dependencies between tokens.

Scaling Laws and Emergence

Kaplan et al. [2] established that language model performance scales as a power-law with model size, dataset size, and compute, spanning more than seven orders of magnitude. Larger models are significantly more sample-efficient. Hoffmann et al. [3] later refined these findings, showing that compute-optimal training requires more data than previously assumed.

A critical phenomenon is the emergence of capabilities at scale: performance on certain tasks remains near-random until a critical threshold, after which it increases substantially [4]. This phase-transition behavior means that modern LLMs exhibit qualitatively different capabilities (reasoning, tool use, multi-step planning) that smaller models simply cannot perform.

Current Landscape

As of early 2026, the major model families include GPT-4/5 (OpenAI), Claude Opus 4.6 and Sonnet 4 (Anthropic), Llama 4 (Meta, open-source), and Mistral Large 3 (Mistral AI, open-weight). These models serve as the **reasoning engine** for AI agents: without tools, they can only generate text; with tools, they become capable of perceiving environments, making decisions, taking actions, and learning from feedback.

Key Insight

LLMs are the “fuel” that powers AI agents. The agent itself is the system built around the LLM: the tool integrations, the orchestration logic, the memory management, and the execution environment. SERGE’s architecture separates these concerns, so the LLM can be swapped while the agent infrastructure remains unchanged.

AI Agents: From ReAct to Modern Architectures

The modern LLM-agent paradigm decomposes autonomous systems into three pillars: **planning**, **memory**, and **tool use** [8]. Agents interleave reasoning with action execution, grounding their decisions in external environments rather than relying solely on parametric knowledge.

Foundational Work: ReAct

Yao et al. [5] introduced the ReAct framework where LLMs generate interleaved *reasoning traces* and *task-specific actions*. Evaluated on HotPotQA, FEVER, ALFWorld, and WebShop, ReAct outperforms pure reasoning or pure acting baselines while improving interpretability. However, ReAct has limitations: single-path reasoning, one LLM call per action, and no separation between

planning and execution.

Beyond ReAct: Modern Agent Paradigms

The field has evolved significantly beyond ReAct. Plan-and-Execute architectures separate planning from execution, reducing LLM calls and improving task completion. Tree of Thoughts [9] introduces search over multiple reasoning paths, while Graph of Thoughts [10] allows arbitrary connections between reasoning steps including aggregation and iterative refinement. ReWOO separates the planning phase entirely from execution, generating a full plan with placeholders before any tool is called.

A recent unified taxonomy [12] organizes agent architectures along six dimensions: perception, brain (core reasoning), planning, action, tool use, and collaboration. Critical design dimensions now include orchestration strategy, memory architecture, and test-time compute scaling, where extended thinking at inference time can be more effective than scaling model parameters.

Claude Code as a State-of-the-Art Agent

The agent we use in this project, Claude Code, represents the current state of the art in agentic systems. Built by Anthropic [15], it implements a single-threaded agentic loop:

```
while(tool_call) → execute tool → feed results → repeat
```

The loop operates in three blended phases: (1) **gather context** by searching files and reading code, (2) **take action** by editing files and running commands, and (3) **verify results** by running tests and checking outputs [16].

Claude Code goes well beyond a simple ReAct loop in several ways [17]:

- **Integrated deep reasoning:** Extended thinking is built into the model, not bolted on as a prompting technique. The model reasons deeply before and during tool use.
- **File-system-native:** The agent operates directly in a project directory with full file system access, git awareness, and terminal execution.
- **Skill-based composition:** Reusable workflows (skills) can be discovered, loaded on demand, and composed. This is more like a library of expert behaviors than a fixed reasoning template.
- **Sub-agent spawning:** The orchestrator-worker pattern is built in, with strict depth limits and clean context boundaries. Each sub-agent runs in its own isolated context window.
- **Context engineering:** Sophisticated management of the finite context window, including compaction, structured notes, and progressive skill loading [17].

Anthropic’s research on building effective agents [15] identifies seven composable patterns: Augmented LLM, Prompt Chaining, Routing, Parallelization, Orchestrator-Workers, Evaluator-Optimizer, and Autonomous Agents. A core principle is to invest in Agent-Computer Interface (ACI) quality, specifically the clarity and documentation of tools exposed to the agent [18].

Tool Use in LLMs

Schick et al. [6] demonstrated with Toolformer that LLMs can learn in a self-supervised fashion to decide which APIs to call, when to call them, and how to incorporate results. Shen et al. [7]

proposed HuggingGPT, a four-stage workflow using an LLM as a controller to orchestrate 24+ specialized models. Patil et al. [11] introduced Gorilla, surpassing GPT-4 on API calls across 1,600+ APIs, along with the Berkeley Function-Calling Leaderboard (BFCL).

Surveys on tool learning [23] identify three dominant paradigms: (1) fine-tuning, (2) in-context learning, and (3) orchestration frameworks. Notably, forcing JSON output for function calls can degrade reasoning performance by over 27 percentage points, highlighting the importance of separating reasoning from tool-call formatting.

Model Context Protocol

The Model Context Protocol (MCP) [20] is an open standard introduced by Anthropic in November 2024 and now hosted by the Linux Foundation. It standardizes how AI agents connect to external data sources and tools through three core primitives: **tools** (callable functions), **resources** (data endpoints), and **prompts** (reusable templates).

MCP solves the “ $N \times M$ ” integration problem: instead of building custom integrations for every combination of agent and tool, both sides implement a single standard protocol. The analogy is USB-C for AI: a universal connector between agents and external systems. MCP is based on JSON-RPC 2.0, inspired by the Language Server Protocol (LSP), and supports both local (stdio) and remote (HTTP/SSE) transports [21].

By early 2025, over 1,000 MCP servers existed in the ecosystem, with SDKs in Python, TypeScript, C#, Java, and Go. Enterprise adoption accelerated rapidly: Microsoft launched a Dynamics 365 ERP MCP Server at Build 2025 [22], Salesforce added native MCP support to Agentforce, and Oracle introduced MCP server support with native database integration.

Why MCP matters for SERGE

MCP is the backbone of SERGE’s architecture. Each supply chain system (SRM, WMS, OMS, TMS) is exposed through MCP tools, providing a standardized interface that *any AI agent* can connect to. It is the agent that connects to MCP, not the LLM directly: the LLM is the reasoning engine inside the agent, and the agent uses MCP tools as its hands and eyes into the supply chain. This means plugging a new agent into our supply chain infrastructure requires **zero changes** to the data access layer.

Supply Chain Management: Fragmentation and Visibility

B2B distribution supply chains are characterized by **fragmented systems**, **data silos**, and **lack of end-to-end visibility**. The SCOR Digital Standard (SCOR-DS) [27], maintained by ASCM, provides a process reference framework covering Plan, Source, Transform, Order, Fulfill, Return, and Orchestrate, emphasizing the need for cross-functional visibility.

Almost two-thirds of companies consider their supply chain a competitive advantage, yet only 6% achieve full visibility [25]. Each enterprise system operates independently: ERP handles finance, OMS manages orders, WMS tracks inventory, and TMS coordinates transport [26]. Without integration, functions like transport ordering become manual processes (email exchanges and hand-keyed responses), creating confusion, errors, and inefficiencies at scale.

Recent industry analysis [28] confirms that most companies run separate standalone systems, creating persistent data silos. Integration through unified approaches can yield 5–10% freight cost savings, yet the rise of marketplace platforms is causing “supplier disintermediation,” forcing distributors to differentiate through data-driven insights [29]. Academic research on breaking down silos [30] proposes integration approaches, but most remain at the infrastructure level (data warehouses, ETL pipelines) rather than offering intelligent query interfaces.

AI and LLM Applications in Supply Chain (2023–2026)

The application of LLMs and AI agents to supply chain management is an emerging field with a sharp research acceleration: from fewer than 5 papers pre-2021 to approximately 45 in 2024 alone. A systematic review of 98 peer-reviewed studies [37] found that roughly 45% do not fully specify their architectures, indicating methodological immaturity. Most evidence remains at prototype level, rarely reporting system-wide KPIs.

Multi-Agent Supply Chain Systems

Jannelli et al. [31] introduced consensus-seeking frameworks where LLM agents represent companies, balancing selfish goals with systemic outcomes. Results show that LLM agents minimize the bullwhip effect better than restocking policies and centralized demand approaches. The authors note that “academic agentic supply chain research is in its infancy.” JD.com deployed the Supply Chain Planning Agent (SCPA) framework [32], demonstrating real-world feasibility.

Diagnostic and Knowledge-Based Approaches

An LLM-driven supply chain diagnosis method [34] integrates LLMs with optimization solvers and fault mode libraries for automated diagnosis. AlMahri et al. [35] propose a Knowledge Graph Query Agent for disruption monitoring. Multi-Agent RAG systems [36] transform unstructured supply chain communications into queryable knowledge bases.

Emerging Paradigms

The concept of Agentic Digital Twins [38] bridges digital twin infrastructure with agentic AI for real-time decision support. The International Journal of Production Research has launched a dedicated special issue on “The Agentic Supply Chain” [39], signaling growing academic recognition. On the industry side, Walmart, Siemens, SAP, Microsoft, and Google are all developing supply chain agent platforms.

LLM-as-Judge Evaluation

LLM-as-Judge uses strong LLMs to evaluate the outputs of other LLMs or systems, providing scalable, cost-effective assessments as an alternative to human expert evaluation. Zheng et al. [40] established the foundational framework with MT-Bench and Chatbot Arena, demonstrating that strong LLM judges achieve **over 80% agreement with human preferences**. Three evaluation modes exist:

- **Pointwise:** Rate a single output on a rubric (our primary mode)

- **Pairwise:** Compare two outputs side by side
- **Listwise:** Rank multiple outputs

Known biases include position bias, verbosity bias, and self-enhancement bias. Comprehensive surveys [41, 42] explore strategies to enhance consistency and mitigate these biases. The EMNLP 2025 taxonomy [43] introduces a three-dimensional framework: *what* to judge, *how* to judge, and *where* to judge.

Relevance to SERGE

SERGE’s evaluation uses LLM-as-Judge in **pointwise mode** with well-defined ground truth. For diagnostic queries across SRM/WMS/OMS/TMS, ground truth is established from known data states, and a judge LLM evaluates whether the agent’s response correctly synthesizes cross-system information. To mitigate self-enhancement bias, we use a different LLM as judge than the one powering the agent.

Research Gaps

Our literature review identifies four significant gaps in the current research landscape:

Research Gap

Gap 1: No standardized supply chain agent benchmarks. Current agent benchmarks (AgentBench [44], SWE-bench, WebArena) focus on software engineering, web navigation, and general reasoning. No benchmark exists for evaluating AI agents operating across supply chain systems.

Research Gap

Gap 2: No plug-and-play architecture connecting SC systems to AI agents. Existing SC-AI solutions are tightly coupled to specific LLM providers or require custom integration code for each supply chain system. Enterprise MCP adoption is recent (2025) and SC-specific MCP servers are not yet standardized.

Research Gap

Gap 3: Read-only diagnostic agents underexplored. Most SC-AI research focuses on optimization (planning, forecasting, replenishment) or autonomous action (consensus-seeking, automated ordering). The read-only diagnostic use case, querying across systems to answer operational questions without modifying data, is underexplored.

Research Gap

Gap 4: No SC-specific evaluation with ground truth. LLM-as-Judge has been validated for general NLP tasks but no evaluation framework exists specifically for supply chain agent responses with domain-specific ground truth.

SERGE directly addresses **Gaps 2, 3, and 4**:

- **Gap 2:** SERGE provides a ready-to-use, plug-and-play architecture via MCP servers for SC systems that any agent can connect to.
- **Gap 3:** SERGE is explicitly a read-only diagnostic agent, filling the underexplored niche.
- **Gap 4:** SERGE's evaluation plan uses LLM-as-Judge with SC-specific ground truth, contributing a novel evaluation methodology.

SERGE can also contribute toward closing Gap 1 by publishing its diagnostic query scenarios as a reusable SC agent evaluation dataset.

Methodology

Overall Approach

Our methodology consists of four components working together:

1. **Four Go services in a monorepo** simulating the supply chain systems (SRM, WMS, OMS, TMS), each with its own data model, repository layer, and controllers.
2. **An MCP integration layer** built with `trpc-mcp-go`, wrapping the service controllers as read-only MCP tools accessible to any AI agent.
3. **An AI agent** (Claude Code) operating in a dedicated project folder, connecting to the MCP servers to perform diagnostic queries.
4. **An LLM-as-Judge evaluation pipeline** comparing agent outputs against predefined ground truth.

To ground the system in a concrete scenario, we model a **small B2B mango distribution company** that sources mangoes from suppliers, stores them in warehouses, processes orders from distributors, and ships via carriers. This deliberately simple use case allows us to model the full supply chain while keeping the data comprehensible for evaluation.

Software Architecture: Four Go Services

The four supply chain systems are implemented as **Go services within a single monorepo**, following a clean three-layer architecture: **structs** (domain entities), **repositories** (data access), and **controllers** (business logic). This is not a microservices architecture; all four services live in the same codebase and share a common build pipeline.

SRM: Supplier Relationship Management

The SRM service manages the upstream supply chain: suppliers, purchase orders, and supplier performance tracking.

Component	Details
Structs	Supplier (ID, name, country, contact, rating, product categories), PurchaseOrder (ID, supplier ID, items, status, order date, expected delivery, actual delivery), SupplierPerformance (supplier ID, on-time rate, avg lead time, defect rate, total POs)
Repository	GetSupplierByID, ListSuppliers, ListSuppliersByCategory, GetPurchaseOrderByID, ListPurchaseOrdersBySupplier, ListPurchaseOrdersByStatus, GetSupplierPerformance, ListOverduePurchaseOrders
Controller	GetSupplier(id) returns full supplier profile with performance metrics. ListPurchaseOrders(filters) returns filtered POs with supplier info. GetSupplierPerformance(id) computes on-time rate, lead time stats, defect rate from PO history. CompareSuppliers(category) returns side-by-side comparison for a product category.

Table 1: SRM service: structs, repository, and controller functions.

WMS: Warehouse Management System

The WMS service tracks products, inventory levels, stock movements, and warehouse locations.

Component	Details
Structs	Product (ID, SKU, name, category, unit, reorder point, reorder quantity), Inventory (product ID, warehouse ID, quantity available, quantity reserved, last updated), StockMovement (ID, product ID, warehouse ID, type [in/out/transfer], quantity, timestamp, reference), Warehouse (ID, name, location, capacity)
Repository	<code>GetProductByID</code> , <code>GetProductBySKU</code> , <code>ListProducts</code> , <code>GetInventory(product, warehouse)</code> , <code>ListInventoryByWarehouse</code> , <code>ListLowStockProducts(threshold)</code> , <code>GetStockMovements(product, dateRange)</code> , <code>ListWarehouses</code>
Controller	<code>CheckInventory(sku)</code> returns current stock across all warehouses with reorder status. <code>GetStockMovements(sku, days)</code> returns recent movements with running balance. <code>GetProduct(id)</code> returns product details with current inventory levels. <code>ListLowStock()</code> identifies products below reorder point.

Table 2: WMS service: structs, repository, and controller functions.

OMS: Order Management System

The OMS service handles customer orders, order lines, and fulfillment status.

Component	Details
Structs	Customer (ID, name, company, segment [retail/wholesale/enterprise], region), Order (ID, customer ID, status [pending/processing/shipped/delivered/cancelled], order date, required date, priority), OrderLine (order ID, product ID, quantity ordered, quantity fulfilled, unit price)
Repository	<code>GetOrderByID</code> , <code>ListOrdersByCustomer</code> , <code>ListOrdersByStatus</code> , <code>ListOverdueOrders</code> , <code>GetOrderLines(orderID)</code> , <code>GetCustomerByID</code> , <code>ListCustomers</code> , <code>GetOrderHistory(customer, dateRange)</code>
Controller	<code>GetOrder(id)</code> returns order with customer info, all lines, and fulfillment status. <code>ListOrdersByCustomer(id)</code> returns order history for a customer. <code>ListOverdueOrders()</code> returns orders past their required date that are not yet delivered. <code>GetOrderLines(id)</code> returns detailed line items with product info and fulfillment rates.

Table 3: OMS service: structs, repository, and controller functions.

TMS: Transport Management System

The TMS service manages shipments, carriers, tracking events, and delivery performance.

Component	Details
Structs	Shipment (ID, order ID, carrier ID, status [pending/in_transit/delivered/exception], origin, destination, shipped date, ETA, actual delivery), Carrier (ID, name, service type, base cost, avg transit time), TrackingEvent (shipment ID, timestamp, location, status, description)
Repository	<code>GetShipmentByID</code> , <code>GetShipmentByOrderID</code> , <code>ListShipmentsByCarrier</code> , <code>ListShipmentsByStatus</code> , <code>GetTrackingEvents(shipmentID)</code> , <code>GetCarrierByID</code> , <code>ListCarriers</code> , <code>GetCarrierPerformance(carrierID)</code>
Controller	<code>GetShipment(id)</code> returns shipment with carrier info and tracking history. <code>TrackDelivery(orderID)</code> finds the shipment for an order and returns full tracking timeline. <code>GetCarrierPerformance(id)</code> computes on-time rate, avg transit time, exception rate. <code>ListExceptionShipments()</code> returns shipments with delivery issues.

Table 4: TMS service: structs, repository, and controller functions.

MCP Integration Layer

The MCP integration layer is built using **trpc-mcp-go** [24], a Go implementation of the Model Context Protocol. It lives in the **same monorepo** as the four Go services.

The architecture follows a clear call chain: the MCP layer receives a tool call from the agent, routes it to the appropriate **controller**, which implements the business logic and calls the **repository** for data access. Each controller function is registered as an MCP tool with typed parameters and descriptions. The MCP layer enforces **read-only access**: only query functions are exposed; no create, update, or delete operations are available to the agent.

MCP Tool	Routes To	Description
<code>srn_get_supplier</code>	SRM Controller	Get supplier profile with performance
<code>srn_list_purchase_orders</code>	SRM Controller	List and filter purchase orders
<code>srn_compare_suppliers</code>	SRM Controller	Side-by-side supplier comparison
<code>wms_check_inventory</code>	WMS Controller	Stock levels across warehouses
<code>wms_get_stock_movements</code>	WMS Controller	Recent stock movement history
<code>wms_list_low_stock</code>	WMS Controller	Products below reorder point
<code>oms_get_order</code>	OMS Controller	Full order with lines and status
<code>oms_list_overdue_orders</code>	OMS Controller	Orders past required date
<code>oms_get_order_lines</code>	OMS Controller	Detailed line items for an order
<code>tms_get_shipment</code>	TMS Controller	Shipment with tracking history
<code>tms_track_delivery</code>	TMS Controller	Find and track shipment for an order
<code>tms_get_carrier_performance</code>	TMS Controller	Carrier on-time and exception rates

Table 5: MCP tools mapped to service controllers.

Following Anthropic’s guidelines on writing tools for agents [18], each MCP tool is designed to be self-contained, clearly documented with example usage, and token-efficient in its responses. The quality of the tool definitions is critical: the better the agent understands what each tool does and when to use it, the more accurately it can reason across systems.

Key Insight

Because the MCP server uses the standard protocol, any MCP-compatible agent can connect to our supply chain system. This provides **plug-and-play capabilities**: swap Claude Code for GPT, Llama, or any other agent, and the supply chain tools remain identical. This is the core of SERGE’s design: a ready-to-use bridge between supply chain infrastructure and AI agents.

AI Agent: Claude Code in a Dedicated Project

For the prototype, we use **Claude Code** as the agent runtime. Claude Code operates in a **dedicated project folder** that defines the agent’s full working context. This folder contains:

- **CLAUDE.md**: The project-level configuration file that instructs the agent on its behavior, available MCP tools, investigation workflows, logging requirements, and constraints (read-only, no data fabrication, always cite sources).
- **Skills**: Reusable workflow definitions (**SKILL.md** files) that encode domain-specific investigation procedures (e.g., `/investigate-order`, `/supplier-report`). Skills are loaded on demand following Anthropic’s progressive disclosure principle [19].
- **Logs**: Structured output directory where the agent’s complete tour (reasoning traces, tool calls, intermediate results, final responses) is captured for evaluation.
- **Reports**: Generated diagnostic reports and analysis outputs.

This folder-based approach provides several advantages. The agent runs in a **containerized environment** where its context, capabilities, and constraints are fully defined. The same folder can be used with different agents (Claude Code, a custom agent via the Claude Agent SDK, or any MCP-compatible agent) without modification. Skills can be extended to add new investigation capabilities without changing the core infrastructure.

Agent Logging for Evaluation

A critical design choice is comprehensive logging. The agent’s complete “tour” for every query (each reasoning step, each MCP tool call, each intermediate result) is captured in structured logs. This tour is then compared against the ground truth by the LLM-as-Judge pipeline. Proper logging guidelines are embedded in the project’s **CLAUDE.md** configuration.

Use Cases

We define **three use cases** representing common operations tasks, deliberately simple and concrete to allow rigorous evaluation:

UC	Name	Systems	Complexity	What It Tests
1	Order Status Inquiry	OMS, TMS	Low	Basic retrieval and data merging from 2 systems
2	Delayed Order Root Cause	OMS, TMS, WMS, SRM	Medium	Cross-system causal chain reasoning (4 systems)
3	Supplier Performance	SRM	Low–Med	Structured data extraction and report generation

Table 6: Three use cases with increasing complexity.

UC-2 is the core value proposition: a human doing this manually would check 4 systems sequentially, each time using the output of the previous query as input for the next. This cross-system reasoning justifies an agentic approach over a simple chatbot.

Evaluation: LLM-as-Judge Pipeline

Our evaluation follows a three-stage pipeline:

1. **Ground truth definition:** For each test scenario, we predefine the correct answer based on the seeded data. We know exactly what the agent *should* find, which systems it should query, and what causal links it should identify.
2. **Agent execution with logging:** The agent processes the query in its dedicated project folder. Its complete tour (reasoning traces + tool calls + responses) is logged.
3. **LLM-as-Judge scoring:** A separate LLM (the “judge”) compares the agent’s response and tour against the ground truth, scoring on predefined criteria.

Judge Evaluation Criteria

The judge LLM scores each agent response on 5 dimensions (1–5 scale):

Criterion	Weight	What the Judge Assesses
Factual correctness	30%	Are all stated facts accurate compared to ground truth?
Completeness	25%	Does the response cover all required data points?
Reasoning quality	20%	Is the causal chain logical and well-explained? (UC-2 mainly)
Source attribution	15%	Does the agent cite which system each piece of data comes from?
Response structure	10%	Is the output well-organized and easy to read?

Table 7: LLM-as-Judge scoring rubric.

Evaluation Protocol

Parameter	Value	Target	Description
Scenarios	15 total		5 per use case, with embedded ground truth
Runs	45 total		3 runs per scenario (consistency measurement)
Judge score	Weighted avg	$\geq 4.0/5.0$	Overall quality threshold
Factual accuracy	Criterion #1	$\geq 90\%$	Core correctness requirement
Scenario pass rate	$\% \geq 3.5$	$\geq 80\%$	Reliability across scenarios
Avg tool calls	Count/query	≤ 6	Efficiency of MCP usage
Response time	Wall clock	< 30 seconds	End-to-end performance
Consistency	Std deviation	$\sigma \leq 0.5$	Stability across repeated runs

Table 8: Quantitative evaluation targets.

Development Timeline

The remaining development is planned over approximately **4 weeks** (March 20 to April 15, 2026):

Week	Dates	Deliverables
1	Mar 20–26	Complete all 4 Go services (structs, repositories, controllers); seed data generation with 15 embedded evaluation scenarios
2	Mar 27–Apr 2	MCP integration layer: register all tools, test tool descriptions; set up Claude Code agent project folder with <code>CLAUDE.md</code> and skills
3	Apr 3–9	Agent integration testing on all 3 use cases; implement LLM-as-Judge pipeline; define judge prompt and scoring rubric
4	Apr 10–15	Full evaluation (45 runs); results analysis and visualization; final report; presentation preparation

Table 9: 4-week development plan (March 20 to April 15, 2026).

Expected Outputs

What We Expect from the Agent

We believe SERGE will **succeed** at the diagnostic tasks defined in our use cases. This is a realistic and achievable goal for the following reasons:

1. **The tasks are well-scoped:** Each use case involves querying a known set of systems with clearly defined data. The agent does not need to handle ambiguity in the data itself, only in the user’s natural language query.
2. **Modern agents excel at tool use:** Claude Code and similar agents have demonstrated strong capabilities in multi-step tool use, reasoning across tool outputs, and generating structured reports. Our use cases (order lookup, root cause analysis, supplier comparison) are well within these capabilities.
3. **The critical success factor is MCP tool design:** The quality of the MCP tool definitions (clear descriptions, well-typed parameters, meaningful return values) directly determines how effectively the agent can use them. We invest significant effort in designing tools following Anthropic’s ACI guidelines [18].

For **UC-1** (Order Status), we expect near-perfect scores: this is a straightforward two-system lookup. For **UC-2** (Root Cause Analysis), the challenge is in the cross-system causal reasoning, but the seeded data creates clear causal chains that a well-prompted agent should follow. For **UC-3** (Supplier Comparison), the main test is structured output quality.

Where We Might Struggle

The main risks are: (1) the agent making unnecessary tool calls when fewer would suffice, (2) inconsistency across repeated runs on the same scenario, and (3) the agent missing one link in a causal chain for UC-2. These are exactly what the evaluation pipeline is designed to measure and quantify.

Project Deliverables

Deliverable	Description	Status
4 Go services (monorepo)	SRM, WMS, OMS, TMS with structs, repositories, controllers	In progress
MCP integration layer	trpc-mcp-go wrapping all 4 services with read-only tools	In progress
Seed data	Realistic mango distributor data with 15 embedded scenarios	In progress
Agent project folder	Claude Code project with <code>CLAUDE.md</code> , skills, and logging	Planned
Evaluation pipeline	LLM-as-Judge with ground truth, scoring rubric, results	Planned
Final report	Complete analysis of results, lessons learned, contributions	Planned

Table 10: Project deliverables and current status.

Research Contributions

Upon completion, SERGE will contribute:

1. **A ready-to-use SC agent architecture:** A documented, reproducible architecture using MCP to connect any AI agent to supply chain systems (Gap 2).
2. **A read-only diagnostic agent prototype:** Demonstrating that cross-system reasoning without write access is a viable and valuable use case (Gap 3).
3. **An SC-specific LLM-as-Judge evaluation:** A methodology for evaluating supply chain agent responses with domain-specific ground truth (Gap 4).
4. **Reusable evaluation scenarios:** 15 diagnostic scenarios with ground truth that can serve as a starting point for SC agent benchmarks (Gap 1).

Knowledge Gained

Beyond the technical deliverables, the team has gained significant knowledge across multiple dimensions:

Supply Chain Domain

- Understanding of the four core enterprise systems (SRM, WMS, OMS, TMS) and their interdependencies
- Awareness of B2B distribution operations: procurement cycles, inventory management, order fulfillment, transport logistics
- Insight into real-world operational pain points: data silos, manual cross-system reconciliation, knowledge concentration in senior staff
- Familiarity with the SCOR Digital Standard and industry frameworks

Software Engineering

- Go programming: clean architecture patterns (structs, repositories, controllers)
- Protocol design: implementing MCP servers, understanding standardized agent-tool communication
- System design: modeling a multi-service architecture with shared data contracts in a monorepo
- Open-source contribution: working with and contributing to `trpc-agent-go` and `trpc-mcp-go`

AI and Agent Engineering

- Deep understanding of agentic AI architectures: from ReAct to modern Claude Code-type agents
- LLM evaluation methodology: LLM-as-Judge, ground truth design, bias mitigation
- Practical experience with agent configuration (`CLAUDE.md`), skill design, and context engineering
- Literature review skills: systematic analysis of 35+ papers across AI agents, SCM, and evaluation

Progress Plan

This section provides an explicit status update and forward plan, designed to also serve as the basis for the midterm presentation slides.

Completed Work (Weeks 1–5)

Week	Phase	What Was Done	Status
1	Topic Selection	Team formation, project proposal, initial architecture concept	Done
2	Domain Research	SC systems research, two-track approach (Go agent vs Claude SDK), tutor meeting	Done
3	State of the Art	14-paper literature review, framework analysis, architecture diagram, Gantt chart	Done
4	Specifications	Advisor feedback response: B2B field, 3 use cases, LLM-as-Judge design, references	Done
5	Implementation Start	Go services scaffold, data model implementation, seed data generation	In Progress

Table 11: Completed milestones.

Remaining Work (March 20 to April 15)

Week	Dates	Key Tasks
1	Mar 20–26	- Complete all 4 Go services with controllers and repositories - Seed database with mango distributor data - Embed 15 evaluation scenarios in seed data
2	Mar 27–Apr 2	- MCP tool registration for all 4 services (read-only) - Set up Claude Code agent project folder with <code>CLAUDE.md</code> - Create investigation skills (<code>SKILL.md</code> files) - Define ground truth for all 15 scenarios
3	Apr 3–9	- Agent integration testing on all use cases - Implement LLM-as-Judge pipeline - Run 45 evaluation runs (3 per scenario) - Collect and analyze results
4	Apr 10–15	- Results analysis and visualization - Final report writing - Presentation preparation - Demo video / live demo

Table 12: Remaining development plan (March 20 to April 15, 2026).

Risk Assessment

Risk	Likelihood	Mitigation
Agent inconsistency across runs	Medium	3 runs per scenario; measure std deviation; tune temperature
MCP server performance	Low	Direct function calls (no network); simple data model
LLM-as-Judge reliability	Medium	Use strong model as judge; validate on subset with human review
Scope creep	Medium	Strict 3 use cases; no write operations; mango distributor only
Poor MCP tool design	Medium	Follow Anthropic ACI guidelines; iterate on tool descriptions

Table 13: Risk assessment and mitigation strategies.

References

- [1] Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. [arXiv:1706.03762](#)
- [2] Kaplan, J. et al. (2020). Scaling Laws for Neural Language Models. [arXiv:2001.08361](#)
- [3] Hoffmann, J. et al. (2022). Training Compute-Optimal Large Language Models. [arXiv:2203.15556](#)
- [4] Wei, J. et al. (2022). Emergent Abilities of Large Language Models. *TMLR*.
- [5] Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*. [arXiv:2210.03629](#)
- [6] Schick, T. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS 2023*. [arXiv:2302.04761](#)
- [7] Shen, Y. et al. (2023). HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. *NeurIPS 2023*. [arXiv:2303.17580](#)
- [8] Weng, L. (2023). LLM Powered Autonomous Agents. [Blog Post](#).
- [9] Yao, S. et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. [arXiv:2305.10601](#)
- [10] Besta, M. et al. (2023). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. [arXiv:2308.09687](#)
- [11] Patil, S. et al. (2024). Gorilla: Large Language Model Connected with Massive APIs. *NeurIPS 2024*. [arXiv:2305.15334](#)
- [12] Agentic Artificial Intelligence: Architectures, Taxonomies, and Evaluation of LLM Agents. (2026). [arXiv:2601.12560](#)
- [13] Large Language Model Agent: A Survey on Methodology, Applications and Challenges. (2025). [arXiv:2503.21460](#)
- [14] Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions. *Artificial Intelligence Review*, Springer, 2025. [Springer](#)
- [15] Anthropic. (2024). Building Effective Agents. [Anthropic Research](#).
- [16] Anthropic. (2025). How Claude Code Works. [Claude Code Documentation](#).
- [17] Anthropic. (2025). Effective Context Engineering for AI Agents. [Anthropic Engineering](#).
- [18] Anthropic. (2025). Writing Tools for Agents. [Anthropic Engineering](#).
- [19] Anthropic. (2025). Equipping Agents with Agent Skills. [Anthropic Engineering](#).

-
- [20] Anthropic. (2024). Model Context Protocol. [Announcement](#). Specification: modelcontextprotocol.io
 - [21] Model Context Protocol Specification (2025-11-25). modelcontextprotocol.io.
 - [22] Microsoft. (2025). Dynamics 365 ERP Model Context Protocol Server. [Microsoft Blog](#).
 - [23] LLM-Based Agents for Tool Learning: A Survey. *Data Science and Engineering*, Springer, 2025. [Springer](#)
 - [24] trpc-mcp-go: Go implementation of the Model Context Protocol. [GitHub](#).
 - [25] GEODIS Supply Chain Visibility Survey. Cited by Descartes, 2024. [Descartes](#).
 - [26] Synchronizing ERP, WMS, and TMS Systems for Supply Chain Optimization. *Supply & Demand Chain Executive / AWS*, 2024.
 - [27] ASCM. SCOR Digital Standard (SCOR-DS). [ASCM](#).
 - [28] Unified Dashboard: Connecting ERP, OMS, WMS, and TMS. Omniful, 2026. [Omniful](#).
 - [29] Challenges & Opportunities in the Trading & Distribution Industry. Globe3. [Globe3](#).
 - [30] Breaking Down Silos: Enhancing Supply Chain Visibility. *IRJMETs*, September 2024.
 - [31] Jannelli, V. et al. (2024). Agentic LLMs in the Supply Chain: Towards Autonomous Multi-Agent Consensus-Seeking. *Int. J. Production Research*, 2025. [arXiv:2411.10184](#)
 - [32] Yin, J. et al. (2025). Rethinking Supply Chain Planning: A Generative Paradigm (SCPA). [arXiv:2509.03811](#)
 - [33] Zheng, G. et al. (2025). LLMs in SCM: Opportunities and a Case Study. *IFAC-PapersOnLine*, MIM 2025.
 - [34] Large Language Model-Driven Supply Chain Diagnosis Method. Springer, 2025. [Springer](#)
 - [35] AlMahri, S. et al. (2026). Automating Supply Chain Disruption Monitoring via an Agentic AI Approach. [arXiv:2601.09680](#)
 - [36] Zhang, Y. et al. (2025). Multi-Agent RAG for Supply Chain Knowledge Bases. KDD 2025 Workshop. [arXiv:2506.17484](#)
 - [37] Systematic Analysis of Generative AI for Supply Chain Transformation. *ScienceDirect*, 2025.
 - [38] Agentic Digital Twins: Bridging Model-Based and AI-Driven Decision-Making. *Int. J. Production Research*, 2026.
 - [39] The Agentic Supply Chain (Special Issue). *Int. J. Production Research*, Taylor & Francis. [Link](#)

- [40] Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS 2023*. [arXiv:2306.05685](#)
- [41] Gu, J. et al. (2024). A Survey on LLM-as-a-Judge. [arXiv:2411.15594](#)
- [42] Li, H. et al. (2024). LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. [arXiv:2412.05579](#)
- [43] Li, H. et al. (2025). From Generation to Judgment: Opportunities and Challenges of LLM-as-a-Judge. *EMNLP 2025*.
- [44] Liu, X. et al. (2024). AgentBench: Evaluating LLMs as Agents. *ICLR 2024*. [arXiv:2308.03688](#)